

IS4 – BDN – T.P. 4

© Polytech Lille

Résumé

Ce TP aborde l'utilisation MongoDB/Python pour les données géographiques.

N'hésitez pas à consulter la documentation en ligne sur le site de MongoDB : <https://docs.mongodb.com/v4.2/reference/> (car la version installée est la 4.2 de MongoDB).

Initialisation du TP

1. Pour ce TP, nous allons utiliser la même base de données `tp` que pour le TP1.
2. Remettez-vous dans votre dossier racine de votre TP1.
3. lancer le daemon dans un shell

```
mongod --dbpath data/db/ &
```

Attention, n'oubliez pas de remettre le lien vers MongoDB dans la variable `$PATH` avant d'utiliser les outils de MongoDB :

(1) `export PATH=$PATH:/usr/localTP/mongo/bin`

(Si à chaque TP vous lancez les même commandes pour étendre le `$PATH`, vous pouvez ajouter la commande (1) directement à la fin du fichier `.bashrc` qui se trouve dans votre répertoire home (taper `kate ~/.bashrc` pour y accéder)).

4. Initialiser conda :

```
/usr/localTP/anaconda3/bin/conda init
```

5. Activer l'environnement `mongoenv` :

```
conda activate mongoenv
```

6. Accéder à la base de données à l'aide d'un script python :

- Écrire dans un fichier externe `nomFichier.py`
- Charger le fichier à l'aide de

```
(mongoenv) $ python3 nomFichier.py
```

7. Testez les importations suivantes dans python pour vérifier que votre environnement marche bien :

```
1 import matplotlib.pyplot as plt
2 from pymongo import MongoClient
3 import pprint
4 import pandas as pd
5 import geopandas
6 import seaborn
7 import contextily
```

I Une première visualisation des données

- I.1. Faites une requête qui affiche les 150 premiers points géographique (contenu dans `address.coord`) en classant par nom et quartier (vous pourrez utiliser `$arrayElemAt` pour récupérer un élément particulier d'un tableau). Vous devriez récupérer une sortie comme cela :

```
[{'borough': 'Manhattan',  
  'cuisine': 'French',  
  'name': '/ L'Ecole',  
  'x': -74.00008199999999,  
  'y': 40.720807},  
 {'borough': 'Manhattan',  
  'cuisine': 'French',  
  'name': '1200 Miles',  
  'x': -73.9921992,  
  'y': 40.7411152},  
 ... ]
```

- I.2. Utiliser les commandes suivantes pour afficher les points avec geopandas :

```
1 import pandas as pd  
2 import geopandas  
3 data = pd.DataFrame(l_resultat)  
4 gdata = geopandas.GeoDataFrame(data, geometry=geopandas.points_from_xy(data.x, data.  
    y), crs="EPSG:4326")  
5 gdata.plot(color="black", figsize=(9, 9), markersize=10)  
6 plt.show()
```

- I.3. Afficher les 200 premiers points, que constatez vous ? Quel est votre hypothèse ?
I.4. Ajouter la carte avec contextily, votre hypothèse est-elle bonne ?

```
1 data = pd.DataFrame(l_resultat)  
2 gdata = geopandas.GeoDataFrame(data, geometry=geopandas.points_from_xy(data.x, data.  
    y), crs="EPSG:4326")  
3 ax = gdata.plot(color="black", figsize=(9, 9), markersize=10)  
4 contextily.add_basemap(ax, crs=gdata.crs.to_string())  
5 plt.show()
```

II Filtre des restaurants

- II.1. Récupérez le fichier `nyc-neighborhoods.geo.json` sur Moodle correspond aux quartiers de New York. Affichez ces quartiers sur la carte en blue en utilisant comme précédemment la méthode `plot` de `nyc`.

```
1 nyc = geopandas.read_file("codes/solutions/nyc-neighborhoods.geo.json")
```

- II.2. Filtrer les points qui sont dans le quartier en utilisant la méthode `sjoin` de `gdata` sur l'objet `nyc` avec les options `how="inner"` et `predicate="within"`.
II.3. Affichez le nombre de point initial et le nombre de points que vous avez filtré.
II.4. Afficher les points que vous avez filtrés sur la carte

III Changer le fond de carte

- III.1. Regardez l'ensemble des cartes disponible dans le providers en utilisant la commande suivante :

```
128 pprint.pprint(contextily.providers)
```

- III.2. Changer le fond de carte de `contextily` en utilisant l'option `source=contextily.providers.XXX` où `XXX` est dans les valeurs suivantes :

```

132 # OpenStreetMap.Mapnik
133 # Esri.DeLorme
134 # Esri.WorldImagery
135 # Esri.WorldShadedRelief
136 # Esri.WorldGrayCanvas
137 # CartoDB.PositronNoLabels
138 # CartoDB.DarkMatter
139 # CartoDB.VoyagerNoLabels
140 # GeoportailFrance.orthos
141 # Strava.All
142 # Strava.Run
143 # Stamen.TonerHybrid
144 # Stamen.TopOSMFeatures

```

IV Afficher la dispersion

IV.1. Afficher la dispersion simple de vos points filtrés :

```

155 g = seaborn.jointplot(x = "x", y = "y",
156                       kind = "scatter", data = gdataj, s=10)
157
158 contextily.add_basemap(
159     g.ax_joint,
160     crs="EPSG:4326",
161     source=contextily.providers.CartoDB.PositronNoLabels,
162 )
163 plt.show()

```

IV.2. Testez les 3 modes kind de seaborn scatter, kde ou hist.

IV.3. Comparer les quartiers en ajoutant l'option hue="borough" à seaborn.

IV.4. On voudrait maintenant comparer la dispersion des cuisines Korean, Caribbean et French.

IV.5. Ajouter en plus le nuage pour plus de visibilité en utilisant la commande :

```

259 g.plot_joint(seaborn.kdeplot, color="r", zorder=0, levels=6 ,fill=True , alpha=0.6)

```

V Et si on faisait quelques requêtes ...

V.1. — Question d'un enseignant : "Parmi les cuisines françaises, afficher les coordonnées des restaurants dont le grade le plus récent ne soit pas 'A'"

— Un étudiant demande "Est-ce que je dois regarder tous les restaurants et garder la note la plus récente qui est différent de 'A' (si elle existe) ou prendre les notes les plus récente de tous les restaurants et ne garder que celles qui sont différentes de 'A' ?"

— L'enseignant répond "Tu vas trouver la même chose !"

Faites les deux requêtes et voyez si vous avez une différence.

V.2. Pour chaque type de cuisine, afficher le restaurant avec la somme de ses scores la plus élevée sur une seule carte.

V.3. Afficher les restaurants à moins de 1 kilomètre du point (-73.93414657, 40.82302903)